ML & NLP ENGINE

Internship Recommendation System

Updated Development Blueprint | Version 2.0

Reflects actual implementation decisions, scoring improvements, and deployment learnings

~ AARYAN RATHOD

What Changed Since Version 1

This updated blueprint reflects every decision made during the actual build. The original blueprint was a planning document. This one documents what was actually built, what was changed during development, what was learned the hard way, and what the final architecture looks like.

Key Changes from v1

Scoring model upgraded from 2 dimensions to 3 (semantic + keyword + location)

Jaccard skill matching replaced with TF-IDF weighted scoring

Default weights changed: semantic 0.60, keyword 0.25, location 0.15

InternshipListing schema expanded with location, country, location_type, domain, stipend, experience_level

SkillWeighter class added to indexer for IDF computation across corpus

CandidateLocationPreference model added to schemas

Streamlit UI added with live weight sliders and location preference controls

Deployment via Docker on Hugging Face Spaces (Python 3.11 locked)

Dataset expanded to 200 rows across 12 domains with structured CSV format

Section 1 -- Technology Stack

The stack below reflects what was actually used in the final build. A few items from the original plan were dropped during deployment due to environment constraints, which are noted explicitly.

1.1 Document Parsing

Library	Purpose and Notes
pdfplumber	Extracts text from PDF files page by page. Handles multi-column resumes and embedded text. When a page returns empty text (scanned PDF), the system detects this and falls back to pytesseract OCR via pdf2image conversion.
python-docx	Parses .docx files natively. Reads paragraphs and table cells. Covers the majority of real-world resume uploads.
pytesseract + pdf2image	OCR fallback for scanned PDFs. Note: these require system binaries (tesseract, poppler). They work locally but need packages.txt on cloud deployments. Removed from Hugging Face build due to binary dependency complexity.
chardet	Detects character encoding before parsing to prevent garbled text from non-UTF-8 files.

1.2 NLP and Entity Extraction

Important deployment note: spaCy works on Python 3.11 and 3.12 but is completely broken on Python 3.14 due to a pydantic v1 compatibility issue inside spaCy internals. Always pin your Python version when using spaCy in production.

Component	Role and Deployment Notes
spaCy (en_core_web_sm)	Named Entity Recognition for candidate name, organization, and date extraction. Use en_core_web_sm for cloud deployments (lower memory). Use en_core_web_trf locally for higher accuracy. The model must be downloaded separately via: python -m spacy download en_core_web_sm
spaCy PhraseMatcher	Rule-based skills detection against a custom taxonomy. Matches 'Python', 'TensorFlow', 'SQL' etc. from a JSON file of known skills. More reliable than NER for skills because skills follow predictable patterns.
NLTK	Stop word removal and text normalization utilities during preprocessing.
langdetect	Detects resume language. Non-English resumes trigger a warning but still process through the pipeline.

1.3 Semantic Embeddings and Vector Search

Library	Usage and Version Notes
sentence-transformers	Core embedding library. Model: all-MiniLM-L6-v2. Produces 384-dimensional vectors. Fast enough for real-time inference.

	normalize_embeddings=True is set so cosine similarity equals dot product, which speeds up FAISS search.
FAISS (faiss-cpu)	Vector index for similarity search. Uses IndexFlatIP (inner product search) for exact results. The index is built offline by build_index.py and loaded at startup. Version note: faiss-cpu 1.13.2 is the correct version for Python 3.14 environments.
PyTorch	Backend for sentence-transformers. Version 2.9.0 for Python 3.14 environments. Version 2.2.2 for Python 3.11 environments. Always check compatibility before pinning.
scikit-learn	Used for cosine similarity in baseline scoring and for evaluation utilities. Use flexible version pin (>=1.3.0) on Python 3.14 to avoid source compilation.
numpy / scipy	Matrix operations for embedding manipulation and distance calculations.

1.4 Scoring Architecture (Updated from v1)

The original v1 used a simple two-signal hybrid score. The final implementation uses three signals with an improved skill scoring approach.

Final Hybrid Score Formula

```
score = (0.60 x semantic_score) + (0.25 x tfidf_skill_score) + (0.15 x location_score)
```

All three weights are user-configurable at runtime through the Streamlit UI sliders. They must sum to 1.0, which is enforced by a Pydantic validator in Settings.

Why TF-IDF Replaced Jaccard

The original Jaccard similarity treated every skill as equally important. This created a problem: a candidate matching on 'Git' or 'Communication' received the same credit as one matching on 'CUDA programming' or 'Federated Learning'. TF-IDF fixes this by computing an IDF weight for each skill across the entire internship corpus. Skills that appear in many listings (common, low-signal) receive low weight. Skills that appear in few listings (rare, high-signal) receive high weight.

```
# IDF formula used in SkillWeighter
idf(skill) = log((1 + N) / (1 + df(skill))) + 1

# where:
# N  = total number of internship listings
# df = number of listings that contain the skill

# Skill overlap score = sum of IDF weights for overlapping skills
```

```
#                        divided by sum of IDF weights for required skills
```

The SkillWeighter is computed during index build time (build_index.py) and saved alongside the FAISS index in the pickle file. It is loaded at startup and passed into rank_recommendations.

Location Scoring Logic

Location preference is an optional third dimension. If the user provides no location preference, a neutral score of 0.5 is returned for all listings, which effectively neutralizes this dimension without breaking the formula.

Condition	Detail	Score
internship is Remote	User has open_to_remote=True	0.95
internship is Remote	User has open_to_remote=False	0.30
City match	preferred_city found in internship location (case-insensitive)	1.00
Country match	preferred_country matches internship country	0.75
Work type match	preferred_location_type matches (Hybrid, On-site)	0.80
No preference set	location_preference is None	0.50 (neutral)
No match	None of the above apply	0.10

1.5 Data Layer

Tool	Role
pandas	Loads and validates internship CSV data during index build. Handles deduplication and column parsing.
SQLAlchemy	Database abstraction layer. Configured for SQLite in development, PostgreSQL in production.
Redis	Caches computed resume embeddings using SHA-256 hash of input text as the key. 1-hour TTL. Cache miss falls back to model inference silently.
Pydantic v2	All data contracts. Request validation, response serialization, settings management. Important: spaCy internally uses pydantic v1, which causes conflicts on Python 3.14.

1.6 API and Frontend

Tool	Role
FastAPI	REST API layer. Auto-generates Swagger UI at /docs. Async request handling. Lifespan context manager loads FAISS index and embedder at startup.
Streamlit	Interactive frontend. Single file (streamlit_app.py) in project root. Imports engine modules directly, does not call FastAPI. Sidebar has sliders for all three scoring weights plus location preference inputs.
Docker	Containerization for deployment. Dockerfile uses python:3.11-slim base image. Runs build_index.py during container build so FAISS binaries are generated inside the container and never committed to git.
pytest	Test suite with unit tests per module and end-to-end integration test. Passed with 0 errors and 2 deprecation warnings on first run.
loguru	Structured logging throughout all pipeline stages.

Section 2 -- Updated Data Schemas

The schemas changed significantly from v1. Below are the final Pydantic models as actually implemented.

2.1 InternshipListing (expanded)

```
class InternshipListing(BaseModel):
    internship_id: str
    title: str
    company: str
    location: str                    # 'San Francisco, CA'
    country: str                     # 'United States'
    location_type: Literal['Remote', 'Hybrid', 'On-site']
    description: str
    required_skills: List[str]       # pipe-separated in CSV, parsed to list
    preferred_skills: List[str] = []
    domain: str = ''                 # 'Machine Learning', 'NLP', etc.
    duration_months: Optional[int] = None
    stipend_usd: Optional[int] = None
    experience_level: Literal['Beginner', 'Intermediate', 'Advanced'] = 'Beginner'
```

2.2 CandidateLocationPreference (new in v2)

```
class CandidateLocationPreference(BaseModel):
    preferred_city: Optional[str] = None
    preferred_country: Optional[str] = None
    preferred_location_type: Optional[Literal['Remote', 'Hybrid', 'On-site']] =
None
    open_to_remote: bool = True
```

2.3 RecommendationResult (expanded)

```
class RecommendationResult(BaseModel):
    internship_id: str
    title: str
    company: str
    location: str
    location_type: str               # added in v2
    domain: str                      # added in v2
    required_skills: List[str]
    match_score: float               # final hybrid score, 0.0 to 1.0
    skill_overlap_pct: float         # TF-IDF weighted overlap
    location_score: float            # added in v2
    stipend_usd: Optional[int]       # added in v2
    explanation: str                 # template-generated, not AI
```

2.4 Settings (updated weights)

```
class Settings(BaseSettings):
    EMBEDDING_MODEL: str = 'all-MiniLM-L6-v2'
    FAISS_INDEX_PATH: str = 'data/faiss_index/index.bin'
    TOP_K_RESULTS: int = 10
    SEMANTIC_WEIGHT: float = 0.60    # was 0.70 in v1
    KEYWORD_WEIGHT: float = 0.25     # was 0.30 in v1
    LOCATION_WEIGHT: float = 0.15    # new in v2
    REDIS_URL: str = 'redis://localhost:6379'
    LOG_LEVEL: str = 'INFO'

    @validator('LOCATION_WEIGHT')
    def weights_must_sum_to_one(cls, v, values):
        total = values.get('SEMANTIC_WEIGHT', 0) + values.get('KEYWORD_WEIGHT', 0)
+ v
        assert abs(total - 1.0) < 0.01, 'Weights must sum to 1.0'
        return v
```

Section 3 -- Architecture Pipeline

The pipeline is unchanged in structure from v1 but stages 5 and 6 have been updated to reflect the new scoring model.

End-to-End Data Flow

1 Resume Input (PDF / DOCX / text) --> File Parser
2 Raw Text --> Preprocessor (unicode, HTML strip, normalization)
3 Clean Text --> NLP Extractor --> CandidateProfile
4 CandidateProfile --> Embedder --> Resume Vector (384-dim)
5 Resume Vector --> FAISS Search --> Top 30 Candidates (semantic only)
6 Top 30 --> Hybrid Scorer (TF-IDF skills + location) --> Re-ranked Results
7 Top-N Results --> Response Formatter --> JSON / Streamlit UI

Note on FAISS search top_k: the index search intentionally retrieves 30 candidates, not the final top_n requested by the user. This gives the hybrid scorer enough candidates to re-rank meaningfully. The final top_n is applied after re-ranking.

3.1 Offline Index Build Process

The FAISS index is never committed to git. It is always built by running build_index.py. This is critical for cloud deployments where binary files cannot be stored in the repository.

```
# Run this before starting the API or Streamlit app
python scripts/build_index.py --input data/internships_batch1.csv

# What build_index.py does:
# 1. Loads and validates each row into InternshipListing schema
# 2. Builds embed_text for each listing:
#    title + company + domain + required_skills + preferred_skills + description
# 3. Embeds all listings in batch using get_embedder().embed_batch()
# 4. Builds FAISS IndexFlatIP from embeddings
# 5. Computes SkillWeighter IDF scores across full corpus
# 6. Saves index (faiss.write_index) and metadata + SkillWeighter (pickle)
```

In the Docker deployment on Hugging Face Spaces, build_index.py runs during the container build (RUN instruction in Dockerfile) so it executes once and the results are baked into the image. This avoids cold-start delays.

Section 4 -- Dataset

4.1 Internship CSV Format

The internship dataset uses a CSV with pipe-separated skills fields. This avoids conflicts between commas in skill names and CSV column delimiters.

```
# Column names (exact, case-sensitive):
internship_id, title, company, location, country, location_type,
description, required_skills, preferred_skills, domain,
duration_months, stipend_usd, experience_level

# required_skills and preferred_skills use pipe separation:
# 'Python|TensorFlow|SQL|NumPy'  -->  ['Python', 'TensorFlow', 'SQL', 'NumPy']
```

4.2 Dataset Distribution (Batch 1, 200 rows)

Property	Details
Domains covered	12 domains: Machine Learning, Data Science, Software Engineering, Web Development, NLP, Computer Vision, DevOps, Cybersecurity, Product Management, Data Engineering, Research, Finance Tech
Location types	40% Remote, 35% Hybrid, 25% On-site
Countries	United States, India, Germany, UK, Canada, Singapore, Australia, Japan, France, Switzerland, and more
Experience levels	Mix of Beginner (30%+), Intermediate, and Advanced
Stipend range	$0 (unpaid) to $3,000 per month
Recommended total size	Run 5 generation batches across different domains to reach 1,000 rows for meaningfully better recommendations

4.3 Recommended Datasets for Model Training

The system uses pre-trained models and does not require training data to work. However, if you want to fine-tune the embedding model or NER model, the following datasets are recommended.

Dataset	Use Case
Resume-NER Dataset (Kaggle)	About 220 annotated resumes with labeled entities: Skills, Education, Name, College. Use this to fine-tune spaCy NER for resume-specific entity recognition.
StackOverflow Developer Survey	Annual survey with 60,000+ self-reported tech skills. Best source for expanding the skills taxonomy beyond the seed list.

LinkedIn Job Postings (Kaggle)	15,000+ job descriptions with required skills. Use for expanding the internship corpus.
JDRA Dataset	Academic dataset pairing resumes with matching and non-matching job descriptions. Ideal for contrastive fine-tuning of the SentenceTransformer model.
Synthetic pairs (recommended)	Generate (resume snippet, internship description) positive/negative pairs using an LLM. Domain-specific synthetic data often outperforms generic datasets because it matches your exact use case.

Section 5 -- Project Structure

The final project structure after all changes. Files marked with * were added in v2.

```
internship_recommender/
├── streamlit_app.py          # Streamlit frontend (imports engine directly)
├── Dockerfile                # python:3.11-slim, builds index during image build
*
├── startup.sh                # Runs build_index then starts Streamlit *
├── requirements.txt          # See Section 6 for version notes
├── packages.txt              # System dependencies for cloud deployment
├── .gitignore                # Includes data/faiss.index and metadata.pkl
├── config.py                 # Pydantic Settings with 3-weight validation
├── api/
│   ├── main.py              # FastAPI app with lifespan loader
│   ├── routes/
│   │   └── recommend.py     # POST /recommend with location support *
│   └── schemas.py           # All Pydantic models (updated for v2) *
├── engine/
│   ├── parser.py            # PDF/DOCX/text parsing with OCR fallback
│   ├── preprocessor.py      # Text cleaning, chunking, language detection
│   ├── extractor.py         # NER + skills extraction (spaCy + regex)
│   ├── embedder.py          # SentenceTransformer wrapper with Redis cache
│   ├── indexer.py           # FAISS index + SkillWeighter *
│   └── scorer.py            # TF-IDF hybrid scoring + location scoring *
├── data/
│   ├── skills_taxonomy.json # 500+ skills seed list
│   └── internships_batch1.csv
├── scripts/
│   └── build_index.py       # Offline index builder (run before API start)
└── tests/
    ├── unit/
    └── integration/
```

Section 6 -- Deployment

This section documents what actually works based on real deployment attempts. Streamlit Cloud was attempted first and failed due to Python 3.14 incompatibility with spaCy. The final working deployment is Docker on Hugging Face Spaces with Python 3.11.

6.1 Why Not Streamlit Cloud

Streamlit Cloud Limitations Encountered

Streamlit Cloud currently defaults to Python 3.14.3 and ignores .python-version files.

spaCy is completely broken on Python 3.14 due to a pydantic v1 internal conflict (pydantic.v1.errors.ConfigError).

faiss-cpu, scikit-learn, scipy, and safetensors all require source compilation on Python 3.14 since wheels are not yet published, causing build timeouts.

Resolution: Use Hugging Face Spaces with Docker and FROM python:3.11-slim to lock the Python version explicitly.

6.2 Hugging Face Spaces Dockerfile

The Dockerfile below is the production version. Key points: Python 3.11 is locked, the FAISS index is built during image build so it is baked in, and the spaCy model is downloaded during build rather than at runtime.

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN python -m spacy download en_core_web_sm

COPY . .

# Build FAISS index from CSV during image build
RUN python scripts/build_index.py --input data/internships_batch1.csv
```

```
EXPOSE 7860

HEALTHCHECK CMD curl --fail http://localhost:7860/_stcore/health

CMD ["streamlit", "run", "streamlit_app.py",
     "--server.port=7860",
     "--server.address=0.0.0.0"]
```

6.3 Requirements.txt Version Notes

Package version pinning is critical and version-dependent. The table below shows what version to use for each Python environment.

Package	Python 3.11 Version	Python 3.14 Version
torch	2.2.2	2.9.0
sentence-transformers	2.7.0	3.4.1
faiss-cpu	1.7.4	1.13.2
scikit-learn	1.4.2	>=1.3.0 (flexible)
numpy	1.26.4	>=1.24.0 (flexible)
scipy	1.13.1	>=1.11.0 (flexible)
spacy	>=3.8.0	BROKEN - do not use
safetensors	0.4.5	>=0.4.0 (flexible)

Rule of thumb: on Python 3.14, never pin C-extension packages (those with Cython or Rust builds) to a specific version. Use >= with a minimum floor and let pip find what has pre-built wheels.

6.4 Git and Binary File Handling

FAISS index files are binary and must never be committed to git when deploying to Hugging Face Spaces. Hugging Face rejects binary files in regular git pushes and requires their Xet storage system instead. The simpler solution is to never commit them at all and always build them at container startup.

```
# Add to .gitignore
data/faiss.index
data/faiss.index.meta.pkl
data/faiss_index/index.bin
```

```
data/faiss_index/metadata.pkl

# If already committed, remove from tracking:
git rm --cached data/faiss.index
git rm --cached data/faiss.index.meta.pkl
git commit -m 'Remove FAISS binaries from tracking'
```

Section 7 -- Development Roadmap

The three-phase roadmap from v1 was completed. Here is what was built in each phase and what remains as future work.

Phase 1: Keyword Baseline *(Completed)*

- File parsers for PDF, DOCX, and plain text
- Text preprocessing pipeline
- spaCy NER entity extraction
- Skills taxonomy PhraseMatcher with 500+ skills
- TF-IDF baseline vectorization with cosine similarity
- CLI script returning top matches
- Unit tests for parser, preprocessor, extractor

Phase 2: Semantic Embeddings *(Completed)*

- SentenceTransformer embeddings (all-MiniLM-L6-v2)
- FAISS IndexFlatIP for vector search
- TF-IDF weighted skill scoring via SkillWeighter
- Location preference as third scoring dimension
- Redis embedding cache
- FastAPI REST endpoint with Pydantic validation
- Streamlit frontend with live weight sliders
- Integration tests passing end-to-end

Phase 3: Production Hardening *(Partially Complete)*

- Docker containerization on Hugging Face Spaces (complete)
- pytest suite with 0 errors, 2 deprecation warnings (complete)

- Edge case handling for empty files, scanned PDFs, non-English (complete)
- Prometheus metrics instrumentation (not yet done)
- Load testing with Locust (not yet done)
- Fine-tuning on domain-specific data (not yet done)
- Expanding dataset to 1,000+ rows across all domain batches (in progress)

Section 8 -- Operational Checklist

Use this checklist every time you set up the project on a new machine or deploy to a new environment.

Before Running Locally

```
[ ]  Python 3.11 active (check with: python --version)
[ ]  pip install -r requirements.txt completed without errors
[ ]  python -m spacy download en_core_web_sm completed
[ ]  data/skills_taxonomy.json exists with 500+ entries
[ ]  data/internships_batch1.csv exists and has correct column names
[ ]  python scripts/build_index.py --input data/internships_batch1.csv completed
[ ]  data/faiss.index file now exists (generated by above command)
[ ]  streamlit run streamlit_app.py launches without import errors
[ ]  Upload a test resume and verify results appear
```

Before Pushing to Hugging Face

```
[ ]  Dockerfile uses FROM python:3.11-slim
[ ]  Dockerfile has RUN python scripts/build_index.py line
[ ]  README.md has correct YAML frontmatter with sdk: docker and app_port: 7860
[ ]  data/faiss.index is in .gitignore
[ ]  data/internships_batch1.csv is committed
[ ]  data/skills_taxonomy.json is committed
[ ]  requirements.txt has Python 3.11 compatible versions
[ ]  git remote huggingface URL uses access token with Write permission
[ ]  git push huggingface main --force succeeds
[ ]  Hugging Face build logs show Python 3.11 and no install errors
```

This document reflects the actual build, not the ideal build.

Every version conflict, deployment failure, and architecture change is documented here so future contributors do not repeat the same debugging sessions.